juliacon

MLThermoProperties.jl: Hybrid Models for Thermodynamic Property Prediction in Julia

Sebastian Schmitt[1], Hans Hasse[1], and Fabian Jirasek[1]

[1]Laboratory of Engineering Thermodynamics (LTD), RPTU Kaiserslautern

ABSTRACT

Hybrid models combining physical knowledge with machine learning gain more and more interest for thermodynamic property prediction. They outperform classical models while being applicable to a much wider scope of chemicals. However, their application in engineering applications stays rare as they are often hardly accessible through unified programmatic interface. To close this gap, we present `MLThermoProperties.jl`, a Julia package that provides a variety of state-of-the-art hybrid models for thermodynamic property prediction. `MLThermoProperties.jl` includes hybrid models for pure component properties like the vapor pressure, mixture properties for predicting phase equilibria, and transport properties. All models only require the SMILES codes of the molecules as input which makes them applicable to nearly all chemical systems in principle. `MLThermoProperties.jl` is built upon the `Clapeyron.jl` package, leveraging its rich thermodynamic solver ecosystem. The `MLThermoProperties.jl` models significantly improve molecular property prediction in various applications in science and engineering, e.g., chemical process engineering, and can be coupled with Julia's rich ecosystem for scientific modeling and simulation. This is demonstrated for a simple example here.

Keywords

Julia, Thermodynamics, Machine Learning, Chemical Engineering

1. Introduction

Knowledge of thermodynamic properties of fluids is crucial in many fields of engineering and science, e.g., for developing new chemical and biotechnological processes, optimizing heat pumps, or designing carbon capture and storage technologies. However, experimental data on thermodynamic properties are notoriously scarce due to the high cost and effort of measurements, making reliable prediction models indispensable. In recent years, machine learning (ML) has emerged as a particularly promising approach to thermodynamic modeling [3].

These models, despite being developed using methods from machine learning, are thermodynamically consistent, i.e., they obey hard physical constraints. This consistency is either achieved by an appropriate architecture [15, 7] or by exploiting existing thermodynamic models to form new hybrid models [17, 5, 6]. Based only on the SMILES code of a substance [20] – a textual representation of the molecular structure – the MLPROP models predict multiple thermodynamic properties, outperforming established models in both accuracy and scope [7]. Among the covered properties are phase equilibria of pure substances and mixtures [16], which are central to chemical process design, as well as transport properties such as diffusion coefficients [17] that govern molecular mass transfer.

Most hybrid models use a chemical representation method to encode the chemical information numerically. Therefore, several techniques exist. Often applied chemical representation methods are chemical language models and molecular graph neural networks (GNNs). Chemical language models are language models operating on the SMILES string, which are pretrained on molecular properties. They deliver a molecular embedding as output which can, without retraining, be used as input for hybrid thermodynamic models (the prediction heads Examples for chemical langua models are ChemBERTa [1] or SMI-TED [14]. A different approach is used by molecular graph neural networks [21]. Here, the molecular structure is encoded in a GNN which is then trained together with the prediction heads directly on the thermophysical properties. This directly encodes the molecular structure in the architecture of the machine learning model.

In Julia, a rich ecosystem for thermodynamics and chemical engineering already exists. `Clapeyron.jl` [19], a mature and comprehensive thermodynamic package, is a central part of this ecosystem. It combines a large number of thermodynamic models (including equations of state (EoS) and Gibbs excess energy (g^E) models) with advanced and efficient solvers. Leveraging Julia's excellent extensibility, several packages extend `Clapeyron.jl`, e.g., `EntropyScaling.jl` [13] for modeling transport properties, or `Langmuir.jl` [12] for modeling adsorption.

This paper introduces `MLThermoProperties.jl`, which provides Julia implementations of hybrid thermodynamic models, based on `Clapeyron.jl` and `EntropyScaling.jl` – enabling the prediction of thermodynamic properties for any substance in Julia. Additionally, the sub-library `ChemBERTa.jl` is presented. An overview of both pacakges is given in 1 Due to the excellent extensibility of Julia packages, `MLThermoProperties.jl` substantially broadens the capabilities for modeling chemical processes, based on state-of-the-art molecular property prediction. The individual models are presented and their integration into the existing thermodynamic ecosystem is explained. Additionally, illustrative applications of these models in chemical process simulations are showcased using packages from the wider scientific-modeling ecosystem in Julia, particularly from the SciML organization.

2. Package overview

`MLThermoProperties.jl` implements several hybrid models and makes them accessible to the thermodynamic solvers from `Clapeyron.jl`. This integration simplifies the implementation of

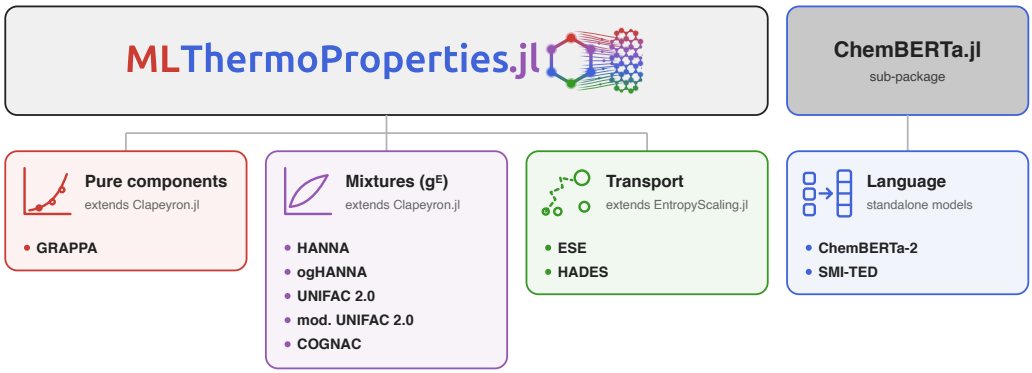


Fig. 1: Overview of the model classes and models in `MLThermoProperties.jl` and its sub-package `ChemBERTa.jl`, and the packages whose property functions they extend.

new models and enables making use of the existing thermodynamic solvers already implemented in `Clapeyron.jl`.

2.1 Models

All currently implemented models were developed under the umbrella of MLPROP [9]. `MLThermoProperties.jl` includes models for pure component properties (see section *2.1.1*), liquid mixture properties, i.e. g^{E} models (see section *2.1.2*), and transport properties (see section *2.1.3*). Additionally, chemical language models are implemented in the sub-pacakge 'ChemBERTa' (see section *2.1.4*). An overview of all model classes and models is given in Figure 1. The package is constantly extended with new models. For a up-to-date list, see the documentation.

2.1.1 Pure component models. GRAPPA is a hybrid GNN for the prediction vapor pressure of pure components, trained on experimental data of about 25,000 components [6]. A graph attention network encodes the molecular graph derived from the SMILES, the only input, and a prediction head yields the parameters of the Antoine equation, from which vapor pressures and boiling points follow at any temperature. Self-diffusion coefficients of pure components are covered by ESE (see section *2.1.3*).

2.1.2 Mixture models. The implemented g^{E} models follow two hybridization strategies [3]: physical constraints are either hard-coded into the architecture of a flexible neural network, or machine learning is used to predict the parameters of an established physical model. All of them provide activity coefficients, from which vapor-liquid equilibria (VLE) can be calculated in combination with a vapor pressure model (e.g., GRAPPA), as well as liquid-liquid equilibria (LLE) and excess properties such as the excess enthalpy h^{E}.
HANNA follows the first strategy: it is a hard-constraint neural network that predicts the excess Gibbs energy g^{E} from the SMILES of the components, the temperature, and the composition [15]. Pure-component embeddings from ChemBERTa-2 (see section *2.1.4*) are processed by a permutation-invariant deep-set architecture, and the output is corrected such that the pure-component limits are fulfilled; the activity coefficients follow by automatic differentiation and thus obey the Gibbs-Duhem equation by construction. Two versions are available: `ogHANNA`, the original model trained on activity coefficients in binary mixtures [15], and `HANNA` (alias `multHANNA`), which is additionally trained on LLE data and excess enthalpies and extended to multi-component mixtures by a geometric projection method [8]. As the neural network is part of the g^{E} model itself, it has to be evaluated at every state point.
UNIFAC 2.0 and mod. UNIFAC 2.0 follow the second strategy and retain the equations of the group-contribution methods UNIFAC and mod. UNIFAC (Dortmund) and only replace their parameter tables [5, 4]. A matrix completion method predicts all pair-interaction parameters between the structural groups, including those missing in the published tables, and is trained end-to-end on experimental activity coefficients (and excess enthalpies for mod. UNIFAC 2.0) from the Dortmund Data Bank. This considerably extends the applicability and improves the accuracy compared to the original versions. Both UNIFAC models are implemented directly in `Clapeyron.jl` as this only requires new parameter tables.
COGNAC [10] also follows the second strategy, using a GNN to predict the parameters of the local-composition framework of mod. UNIFAC for individual atoms instead of predefined structural groups [10]. A message-passing neural network encodes each atom together with its molecular neighborhood, from which two prediction heads yield atomic volume and surface-area parameters as well as interaction parameters for every atom pair. These parameters enter the unchanged mod. UNIFAC equations, so that thermodynamic consistency is guaranteed and the neural network has to be evaluated only once per set of components rather than at every state point. COGNAC was trained on about 820,000 experimental data points comprising VLE, LLE, solid-liquid equilibria (SLE), activity coefficients at infinite dilution, and excess enthalpies.

2.1.3 Transport models. Both transport models are based on the Stokes-Einstein (SE) equation, whose prediction is multiplied by a positive scaling factor obtained from a neural network that takes molecular descriptors derived from the SMILES as input. As the scaling factor does not depend on temperature, the physically consistent temperature dependence of the SE equation is retained.
ESE (Enhanced Stokes-Einstein) predicts diffusion coefficients D_{ij}^{∞} of a solute i at infinite dilution in a pure solvent j [17]. The only inputs are the SMILES of solute and solvent and the solvent viscosity at the temperature of interest, which can be provided by any viscosity model of `EntropyScaling.jl`. Choosing the solute as solvent ($j = i$) yields the self-diffusion coefficient of the pure component.
HADES extends ESE to composition-dependent self-diffusion coefficients in liquid mixtures with any number of components, covering the entire range from infinite dilution to the pure components

[18]. The descriptors of the solvent components are aggregated by an equivariant deep-set model, which ensures invariance with respect to their permutation, and the viscosity of the mixed solvent is obtained by logarithmic mixing of the pure-component viscosities. HADES was trained on 2,526 experimental data points for 600 systems; besides the SMILES, only the pure-component viscosities are required as input.

2.1.4 Chemical language models. In `ChemBERTa.jl`, two language models are available: ChemBERTa [1] and SMI-TED [14]. ChemBERTa-2 is a BERT-type transformer that was pretrained on the SMILES of 77 million molecules from PubChem by regression of molecular descriptors [1]. `ChemBERTa.jl` provides the variant used in HANNA, whose tokenizer was fixed by Specht et al. [15]; the embedding of the classification token of the last layer serves as a 384-dimensional representation of the molecule. Before tokenization, the SMILES are canonicalized with RDKit, either via `RDKitMinimalLib.jl` or via `PythonCall.jl`. SMI-TED is a transformer-based encoder-decoder foundation model with 289 million parameters that was pretrained on 91 million curated SMILES from PubChem by masked token prediction and reconstruction of the SMILES from its latent space [14]. The encoder condenses the token embeddings into a fixed-size latent representation of the molecule, which, owing to the reconstruction objective, separates molecules according to chemically relevant features and serves as molecular embedding. The decoder, a non-autoregressive feed forward decoder, can be used to generate a SMILES from a given embedding. Both models are implemented natively in Julia based on `Lux.jl` in the registered sub-package `ChemBERTa.jl`, which can also be used independently, e.g., as molecular input representation for new hybrid models.

2.2 Integration in Julia's Thermodynamic Ecosystem

All models are implemented by extending a property function from `Clapeyron.jl` or `EntropyScaling.jl` and making use of existing thermodynamic solvers. This is exemplarily explained for the `HANNA` model and the `ESE` model in the following. For the Gibbs excess energy model `HANNA`, a custom function is defined which calculates the Gibbs excess energy G^E through a neural network (see Code 2). This is the only thermodynamic property that needs to be defined. Derived quantities like the activity coefficients γ_i and the excess enthalpy H^E can be calculated based on the HANNA model by Clapeyron's property functions, i.e. `Clapeyron.activity_coefficient(model::HANNA,p,T,z)` and `Clapeyron.excess(model::HANNA,p,T,z,enthalpy)`, without further ado. This exploits the universal thermodynamic relations between those properties which can be calculated as derivations of G^E, i.e.

$$\ln \gamma_i = \frac{1}{RT} \left(\frac{\partial G^E}{\partial n_i} \right)_{T,p,n_{j\neq i}} \tag{1}$$

$$H^E = -RT^2 \left(\frac{\partial \left(G^E/RT \right)}{\partial T} \right)_{p,\boldsymbol{n}} . \tag{2}$$

These universal thermodynamic relations are already implemented in `Clapeyron.jl`. The required derivatives are evaluated by automatic differentiation.

Code 1: Simplified code for integrating HANNA [15] in `Clapeyron.jl`.

```
Clapeyron.excess_gibbs_free_energy(
    model::ogHANNA, p, T, z
) = begin
    x = z ./ sum(z)

    params = model.params
    Ts = scale(params.scaler_T, T)
    gE = params.nn((Ts,x,params.θs.values))

    return gE * Rgas(model) * T * sum(z)
end
```

The `ESE` model follows the same principle, but extends `EntropyScaling.jl` instead. The only method implemented evaluates the Stokes-Einstein equation corrected by a factor b_{ij}, which a neural network ensemble predicts from molecular descriptors of solute i and solvent j [17], i.e.

$$D_{ij}^{\infty} = b_{ij} \frac{k_\mathrm{B} T}{6\pi \, \eta_j(p,T) \, r_i} \tag{3}$$

with the Boltzmann constant k_B and the hydrodynamic radius r_i of the solute. Since b_{ij} depends on the components only, it is evaluated once when the model is constructed and the temperature dependence is dictated entirely by the Stokes-Einstein equation, which enforces physical consistency. The solvent viscosity η_j is obtained by calling `EntropyScaling.viscosity(model,p,T)` on a viscosity model supplied by the user, so that any model of `EntropyScaling.jl` can be combined with `ESE`, e.g., a substance-specific reference correlation (`RefpropRES`), a group-contribution entropy scaling model (`GCES`), or a constant experimental viscosity (`ConstantModel`). Based on this single method, `EntropyScaling.inf_diffusion_coefficient` is readily available for `ESE`, returning the diffusion coefficients of all component pairs or of an individual solute-solvent pair.

Code 2: Simplified code for integrating ESE [17]in `EntropyScaling.jl`.

```
EntropyScaling._inf_diffusion_coefficient(
    model::ESE, p, T, (idx_i,idx_j); phase=:liquid
) = begin
    f = 0.64
    ϱ_ref = 1050.
    M_i = model.params.Mw.values[idx_i]*1e-3
    pure_vismodel = model.pure_vismodels[idx_j]
    η_j = viscosity(pure_vismodel, p, T; phase)

    r_i = cbrt(f * 3 * M_i / (4π * ϱ_ref * NA))
    Dij_SE = (kB*T)/(6π*η_j*r_i)
    return Dij_SE * model.params.b_ij[idx_i,idx_j]
end
```

The syntax to construct the models follows `Clapeyron.jl`'s and `EntropyScaling.jl`'s syntax closely. Existing models from both packages can be used in combination with the new hybrid models, e.g., the g^E models can be used with any pure component model (defined through the keyword argument `puremodel`), and in ESE a viscosity model can be used that directly enables the prediction of the temperature dependency.

3. Example Application

3.1 Calculating Mixture Properties

The following example computes the isothermal p-xy diagram of the system tetrachloromethane (1) + *tert*-butanol (2) at $T = 298.15$ K, using `HANNA` for the activity coefficients and `GRAPPA` for the pure-component vapor pressures (Listing 3). Both models are constructed from the component names only, which are resolved to SMILES through the `Clapeyron.jl` database. The vapor-liquid equilibrium is calculated with the γ-φ approach; assuming an ideal gas phase and neglecting the Poynting correction, it reduces to the extended Raoult's law

$$p\, y_i = x_i\, \gamma_i(\boldsymbol{x}, T)\, p_i^{\mathrm{s}}(T), \tag{4}$$

where x_i and y_i are the mole fractions of component i in the liquid and the vapor phase, respectively. The activity coefficients γ_i are supplied by HANNA and the vapor pressures p_i^{s} by GRAPPA, while the phase-equilibrium solver is taken from `Clapeyron.jl`. Since the pure-component model is a free choice, GRAPPA can be replaced by a cubic or multiparameter EoS without changing the rest of the code. The resulting phase diagram is shown in Figure 2.

Code 3: Calculation of the p-xy diagram of tetrachloromethane (1) + *tert*-butanol (2) at 298.15 K with HANNA and GRAPPA.

```julia
using MLThermoProperties, Clapeyron, CairoMakie

comps = ["tetrachloromethane", "tert-butanol"]
model = multHANNA(comps; puremodel=GRAPPA)

T = 298.15
x₁ = range(0.0, 1.0, 51)
p, y₁ = zeros(51), zeros(51)
lnγ₁, lnγ₂ = zeros(51), zeros(51)
for (i, x) in enumerate(x₁)
    z = [x, 1-x]
    pᵢ, _, _, yᵢ = bubble_pressure(model, T, z)
    p[i], y₁[i] = pᵢ/1e5, yᵢ[1]
    γᵢ = activity_coefficient(model, 1e5, T, z)
    lnγ₁[i], lnγ₂[i] = log.(γᵢ)
end

# Plotting ...
fig = Figure(size=(400, 500), fontsize=14)
ax1 = Axis(fig[1,1]; limits=((0,1), nothing),
    xticklabelsvisible=false,
    ylabel=L"p / \mathrm{bar}")
ax2 = Axis(fig[2,1]; limits=((0,1), nothing),
    xlabel=L"x_1,y_1 / \mathrm{mol\,mol^{-1}}",
    ylabel=L"\ln \gamma_i")
lines!(ax1, x₁, p; color=:blue, label="bubble")
lines!(ax1, y₁, p; color=:red, label="dew")
axislegend(ax1; position=:lt, framevisible=false)
lines!(ax2, x₁, lnγ₁; color=:navy, label="γ₁")
lines!(ax2, x₁, lnγ₂; color=:green, label="γ₂")
axislegend(ax2; position=:lt, framevisible=false)
save(joinpath(@__DIR__, "fig_hanna.pdf"), fig)
```

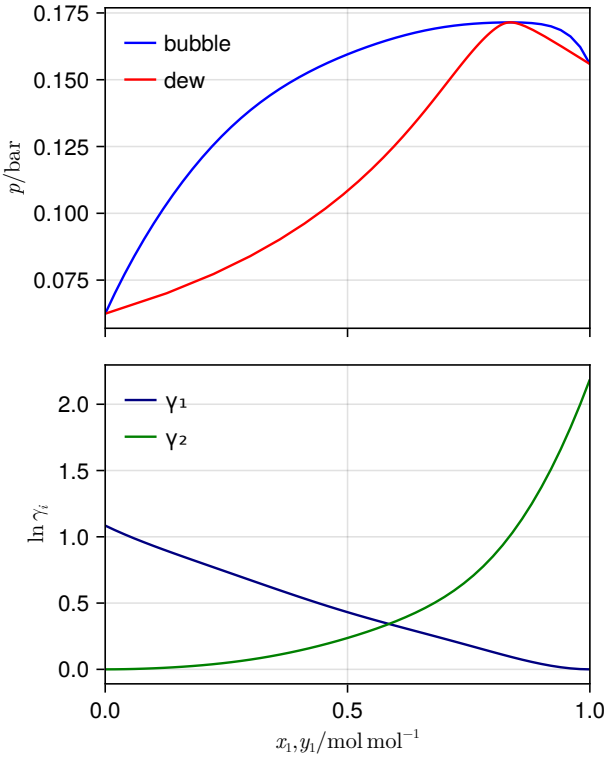


Fig. 2: Predicted p-xy diagram of tetrachloromethane (1) + *tert*-butanol (2) at $T = 298.15$ K, obtained with HANNA (activity coefficients) and GRAPPA (pure-component vapor pressures).

3.2 Diffusion Coefficients

Transport properties are accessed through `EntropyScaling.jl`. Code 4 computes the diffusion coefficients at infinite dilution of ethanol in *n*-decane and of *n*-decane in ethanol between 280 K and 340 K. The `ESE` model is again built from the component names alone; the solvent viscosity required by the Stokes-Einstein equation is calculated from the default `RefpropRES` correlations. Alternatively, other `EntropyScaling.jl` models, e.g., group-contribution models or simply constant viscosity, can be used by passing them via the `vismodel` keyword. The two curves in Figure 3 differ by about one order of magnitude, which reflects the different viscosities of the two solvents.

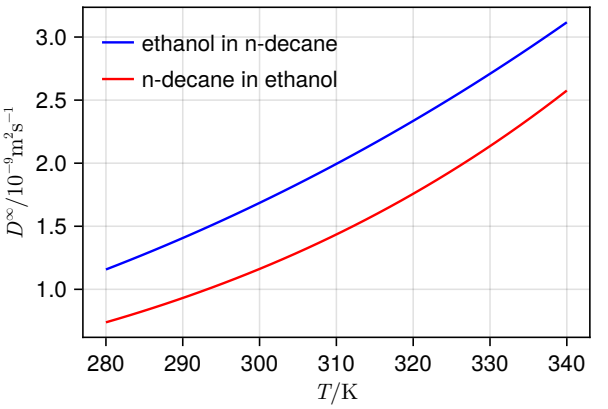


Fig. 3: Predicted diffusion coefficients at infinite dilution of ethanol in *n*-decane and of *n*-decane in ethanol at $p = 1$ bar, obtained with ESE and the `RefpropRES` viscosity correlations.

Code 4: Diffusion coefficients at infinite dilution in the system ethanol + *n*-decane with ESE.

```julia
using MLThermoProperties, EntropyScaling
using CoolProp, CairoMakie

model = ESE(["ethanol", "n-decane"])

p = 1e5
T = range(280.0, 340.0, 40)
D₁₂ = inf_diffusion_coefficient.(model, p, T;
    solute="ethanol", solvent="n-decane")
D₂₁ = inf_diffusion_coefficient.(model, p, T;
    solute="n-decane", solvent="ethanol")

# Plotting ...
fig = Figure(size=(400, 280), fontsize=14)
ylabel = L"D^\infty / 10^{-9}\mathrm{m^2 s^{-1}}"
ax = Axis(fig[1,1]; xlabel=L"T / \mathrm{K}",
    ylabel)
lines!(ax, T, D₁₂ .* 1e9; color=:blue,
    label="ethanol in n-decane")
lines!(ax, T, D₂₁ .* 1e9; color=:red,
    label="n-decane in ethanol")
axislegend(ax; position=:lt, framevisible=false)
save(joinpath(@__DIR__, "fig_ese.pdf"), fig)
```

3.3 Chemical Process Simulation

The hybrid models implemented in `MLThermoProperties.jl` can easily be coupled with Julia's modeling and simulation ecosystem. This is particularly interesting for, e.g., applications in chemical engineering, where the simple access to thermodynamic models is crucial. An important task in chemical engineering is the modeling and design of batch distillation processes to separate a given mixture. Batch distillation columns can be modelled by the equilbrium stages concept where every stage is described by the MESH equations [2] which include the mass balances (M), chemical equilbrium (E), the summation condition (S), and energy balance (H for enthalpy). A Julia implementation of batch distillation simulations based on the MESH equations is given in github.com/se-schmitt/BatchDistillation.jl. The implementation is based on `ModelingToolkit.jl` [11], a framework for symbolic-numeric equation-based modeling. `ModelingToolkit.jl` is widely applied for acausal modeling tasks.

As an example, the separation of the mixture methyl benzoate + ethyl benzoate + dimethyl phthalate in a batch distillation column with 12 (theoretical) stages is simulated. The thermodynamic model is assembled with `Clapeyron.jl`'s `CompositeModel`: COGNAC provides the activity coefficients, GRAPPA the vapor pressures, the DIPPR 105 correlation the liquid densities, and the Tsonopoulos virial correlation the gas phase. The reboiler is charged with 20 mol of the mixture ($x_i = \{0.5, 0.3, 0.2\}$) and heated with a constant duty of 300 W at a pressure of 10 kPa. Half of the condensed vapor is withdrawn as distillate, the rest is returned to the column as reflux. Starting from steady state at total reflux, the differential-algebraic system generated by `ModelingToolkit.jl` is integrated until the reboiler runs dry or one component is purified.

The results are shown in Figure 4. The compositions in the rebolier and the distillate vessel are shown as a function of the time in the left part of the figure. The light boiler methyl benzoate is removed first and dominates the distillate, followed by ethyl benzoate, while the heavy boiler dimethyl phthalate accumulates in the reboiler until it is obtained in pure form after about 100 min. Accordingly, the temperature in the column rises from about 400 K to 470 K, with a steep temperature front moving through the column as ethyl benzoate is depleted. Additionally, the temperature distribution in the distillation column is plotted for several times. As expected, due to the advancing removal of the lighter boiling component, the temperature increases with over time in all stages. While in the beginning and at the end of the simulation all stages have a similar temperature, a sharp temperature gradient in the distillation column can be seen inbetween. The full script for the simulation is given in the supplementary material.

Code 5: Batch distillation simulation.

```julia
## setup
using Printf, Statistics, ADTypes, CairoMakie
using ModelingToolkit, SciMLBase, OrdinaryDiffEq
using Clapeyron, MLThermoProperties
using BatchDistillation

# input
n_dry  = 1.      # mol, mininmum reboiler holdup
x_pure = 1e-4    # mol mol⁻¹, purity criteria

# system
components = [
    "methyl benzoate",
    "ethyl benzoate",
    "dimethylphthalate"
]

# thermodynamic model
model = CompositeModel(components;
    liquid=COGNAC,
    fluid=CompositeModel(components;
        liquid=DIPPR105Liquid,
        saturation=GRAPPA,
        gas=TsonopoulosVirial
    )
)

# plant-specific parameters
N_stages = 12
H_col    = 1.5  # m, height of column

# experiment-specific parameters
Q    = 300.     # W, reboiler heat duty
PS   = 10000.0  # Pa, pressure in column
ϵ    = 0.50     # efflux ratio
x_charge  = [0.5, 0.3, 0.2] # reboiler charge
nB   = 3.3      # mol, buffer holdup
n_charge = 20   # mol, reboiler charge

## run
# create column
@named col = Column(; fluid=model, N_stages)
sys = mtkcompile(col)

# parameter map
params = Dict(sys.reb.p => PS, sys.reb.Q => Q,
    sys.cond.p => PS, sys.cond.nB => nB,
    sys.cond.ϵ => ϵ,
    (getproperty(sys, Symbol(:stage_, j)).p => PS
        for j in 2:N_stages)...,
)

# initialization
init = initialize_total_reflux(sys;
    fluid=model, N_stages,
    p=PS, Q, x_charge, n_charge
)

params0 = merge(init.u0, params)
```

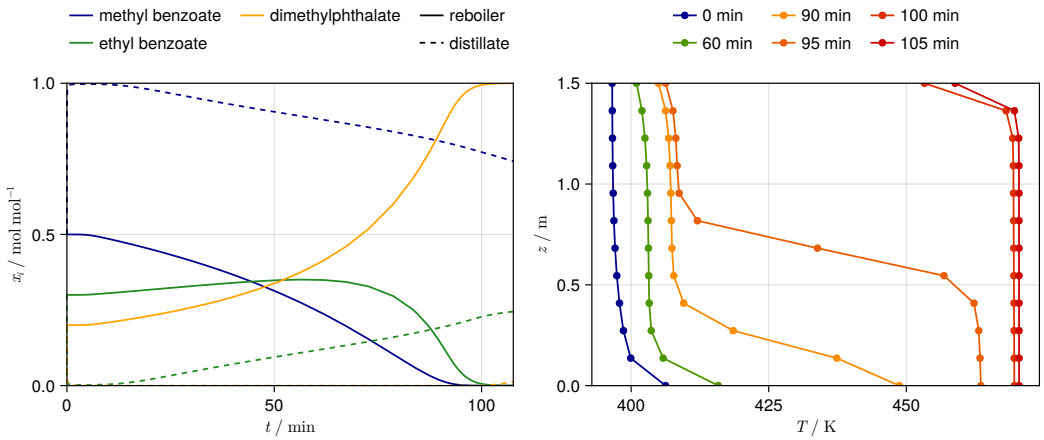


Fig. 4: Simulated batch distillation of methyl benzoate + ethyl benzoate + dimethyl phthalate at $p = 10$ kPa, obtained with COGNAC (activity coefficients) and GRAPPA (vapor pressures). Left: mole fractions x_i in the reboiler (solid lines) and the distillate vessel (dashed lines) as a function of time t. Right: temperature profiles over the column height z at selected times.

```
prob = ODEProblem(sys, params0, (0.0, 7200.);
    guesses = init.guesses
)

# callbacks/events
reb_nL = ModelingToolkit.getu(sys, sys.reb.nL)
reb_x = ModelingToolkit.getu(sys,
    collect(sys.reb.x)
)
_dryout = DiscreteCallback(
    (u,t,i) -> reb_nL(i) < n_dry ||
        maximum(reb_x(i)) > 0.9999,
    terminate!; save_positions = (true, false)
```

4. Conclusions

`MLThermoProperties.jl` makes hybrid thermodynamic models that combine machine learning with physical knowledge available in Julia. By implementing each model as an extension of a single property function of `Clapeyron.jl` or `EntropyScaling.jl`, the models inherit the complete solver ecosystem of these packages: activity coefficients, excess properties, and phase equilibria follow from one machine-learned expression for the excess Gibbs energy, and derived properties are obtained by automatic differentiation without analytical derivatives. This simplifies the calculation thermophysical properties with these hybrid models. Additionally, it enables the application of the hybrid models in complex simulations, e.g. for applications in chemical engineering.
Moreover, the sublibrary `ChemBERTa.jl` provides implementations of chemical language models, in particular ChemBERTa and SMI-TED. These models can beneficially used in many areas of cheminformatics to embed chemical information of a molecule only based on the SMILES of the molecule.
The package is constantly extended by new hybrid models. The models implemented here are also accessible through the MLPROP web interface [9], which is aimed at quick interactive calculations of single systems. `MLThermoProperties.jl` complements this by making the same models available inside automated workflows: they can be called in loops, embedded in process models, fitted to, differentiated through, and combined with the wider Julia ecosystem for scientific modeling and simulation, as demonstrated above.

Acknowledgements

[HIER NOCH ERGÄNZEN]

Code availability

5. References

[1] W. Ahmad, E. Simon, S. Chithrananda, G. Grand, and B. Ramsundar. ChemBERTa-2: Towards Chemical Foundation Models, 2022. doi:10.48550/arXiv.2209.01712.

[2] A. Gorak and E. Sorensen. *Distillation: fundamentals and principles*. Elsevier, London Waltham San Diego, 2014.

[3] H. Hasse, S. Schmitt, and F. Jirasek. Artificial intelligence in thermodynamics: hybrid modeling of thermophysical properties of fluids. *Current Opinion in Chemical Engineering*, 51:101236, 2026. doi:10.1016/j.coche.2026.101236.

[4] N. Hayer, H. Hasse, and F. Jirasek. Modified UNIFAC 2.0-A Group-Contribution Method Completed with Machine Learning. *Industrial & Engineering Chemistry Research*, 64(20):10304–10313, 2025. doi:10.1021/acs.iecr.5c00077.

[5] N. Hayer, T. Wendel, S. Mandt, H. Hasse, and F. Jirasek. Advancing thermodynamic group-contribution methods by machine learning: UNIFAC 2.0. *Chemical Engineering Journal*, 504:158667, 2025. doi:10.1016/j.cej.2024.158667.

[6] M. Hoffmann, H. Hasse, and F. Jirasek. GRAPPA—A hybrid graph neural network for predicting pure component vapor pressures. *Chemical Engineering Journal Advances*, 22:100750, 2025. doi:10.1016/j.ceja.2025.100750.

[7] M. Hoffmann, T. Specht, Q. Göttl, J. Burger, S. Mandt, H. Hasse, and F. Jirasek. A machine-learned expression for the excess Gibbs energy, 2025. doi:10.48550/arXiv.2509.06484.

[8] M. Hoffmann, T. Specht, Q. Göttl, J. Burger, S. Mandt, H. Hasse, and F. Jirasek. Thermodynamically consistent machine learning model for excess Gibbs energy. *Nature Communications*, 17(1):3485, 2026. doi:10.1038/s41467-026-71430-y.

[9] M. Hoffmann, T. Specht, N. Hayer, H. Hasse, and F. Jirasek. MLPROP - An Interactive Web Interface for Thermophysical Property Prediction with Machine Learning. *Chemie Ingenieur Technik*, 97(11-12):1052–1056, 2025. doi:10.1002/cite.70004.

[10] Marco Hoffmann, Sebastian Schmitt, Hans Hasse, and Fabian Jirasek. Learning atomic interactions for the thermodynamics of liquid mixtures. in preparation, 2026.

[11] Y. Ma, S. Gowda, R. Anantharaman, C. Laughman, V. Shah, and C. Rackauckas. ModelingToolkit: A Composable Graph Transformation System For Equation-Based Modeling, 2021. doi:10.48550/ARXIV.2103.05244.

[12] V. V. Santana, A. Riedemann, T. Christiansson, and P. J. Walker. Langmuir.jl.

[13] S. Schmitt, A. Riedemann, E. B. D. M. Alves, and LéoSchmider. se-schmitt/EntropyScaling.jl: v0.3.5, 2025. doi:10.5281/zenodo.17097480.

[14] E. Soares, E. Vital Brazil, V. Shirasuna, D. Zubarev, R. Cerqueira, and K. Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8(1):193, 2025. doi:10.1038/s42004-025-01585-0.

[15] T. Specht, M. Nagda, S. Fellenz, S. Mandt, H. Hasse, and F. Jirasek. HANNA: hard-constraint neural network for consistent activity coefficient prediction. *Chemical Science*, 15:19777–19786, 2024. doi:10.1039/D4SC05115G.

[16] T. Specht, M. Nagda, S. Fellenz, S. Mandt, H. Hasse, and F. Jirasek. HANNA: Hard-constraint Neural Network for Consistent Activity Coefficient Prediction, 2024. doi:10.48550/arXiv.2407.18011.

[17] J. Wagner, Z. Romero, K. Münnemann, S. Schmitt, T. Specht, H. Hasse, and F. Jirasek. Hybrid machine learning for enhanced prediction of diffusion coefficients in liquids. *Physical Chemistry Chemical Physics*, 28(40):25407–25416, 2026. doi:10.1039/d6cp00793g.

[18] J. Wagner, T. Specht, H. Hasse, and F. Jirasek. Composition-Dependent Self-Diffusion Coefficients in Liquid Mixtures from Hybrid Machine Learning, 2026. doi:10.48550/arXiv.2609.24599.

[19] P. J. Walker, H.-W. Yew, and A. Riedemann. Clapeyron.jl: An Extensible, Open-Source Fluid Thermodynamics Toolkit. *Industrial & Engineering Chemistry Research*, 61(20):7130–7153, 2022. doi:10.1021/acs.iecr.2c00326.

[20] D. Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi:10.1021/ci00057a005.

[21] O. Wieder, S. Kohlbacher, M. Kuenemann, A. Garon, P. Ducrot, T. Seidel, and T. Langer. A compact review of molecular property prediction with graph neural networks. *Drug Discovery Today: Technologies*, 37:1–12, 2020. doi:10.1016/j.ddtec.2020.11.009.